Disjoint sets implementations: linked lists and rooted trees

Angela Liguori

Contents

1 Introduction **2**
1.1 Problem statement 2
1.2 Hardware and software specifications 2

2 Theoretical analysis of the problem **2**
2.1 Fundamental Aspects 2
2.1.1 Disjoint sets 2
2.1.2 Connected components 3
2.2 Linked lists 3
2.2.1 Implementation of disjoint sets as linked lists 4
2.3 Forests of rooted trees 4
2.3.1 Implementation of disjoint sets as rooted trees 4
2.4 Assumptions and hypotheses 5

3 Code documentation **6**
3.1 Content outline and interaction between modules 6
3.2 Analysis of implementation choices 7
3.3 Description of implemented methods 7

4 Tests and results **9**
4.1 Data 9
4.2 Evaluation metrics 9
4.3 Results analysis 10

5 Conclusion **12**

6 References **14**

1 Introduction

1.1 Problem statement

We will compare different methods to implement data structures for disjoint sets through:

- **linked lists**, with and without the weighted union heuristic,
- **rooted trees forests**, using the path compression heuristic.

The comparison will be based on the efficiency of algorithms to find connected components in undirected graphs.

1.2 Hardware and software specifications

The results of the tests will be obtained as allowed by the computational restrictions due to the following characteristics, specific to the device on which they will be run.

- **CPU**: AMD 3020e with Radeon Graphics 1.20 GHz
- **RAM**: 8 GB
- **OS**: Windows 10 IoT Enterprise
- **Python version**: 3.8.10

This report is written in LateX, using the Overleaf online editor.

2 Theoretical analysis of the problem

In order to understand the nature of the comparison between the structures mentioned above, every theoretical aspect of the subject will be covered in this section.

2.1 Fundamental Aspects

2.1.1 Disjoint sets

Two sets A and B are considered **disjoint** if their intersection is the empty set: $A \cap B = \varnothing$.

A disjoint set data structure maintains a family $S = \{S_1, S_2, ..., S_k\}$ of dynamic disjoint sets, each identified by a **representative**, i.e. an element of the set itself. The choice of said element varies, depending on the application and implementation of the data structure.

x and y being objects representing elements of a set, these are fundamental operations a disjoint set data structure must support:

- **make_set(x)**: where x previously belongs to no set, creates a new set, consisting of the sole element x, which is also set as representative;
- **find_set(x)**: returns a pointer to the representative of the set to which x belongs;
- **union(x,y)**: joins two dynamic disjoint sets, each containing either x or y, together into a new joint set, which is the union of the two. Doing so entails two necessary other operations: destroying the two separate sets from the family of sets and changing the representative to any one element of the newly created set.

2.1.2 Connected components

In a **graph** $G = (V, E)$, the vertices $u, v \in V$ belong to the same **connected component** if and only if there is a path between them.

One of many applications for disjoint sets data structures is to determine the connected components in undirected graphs.

The method **connected_components(G)** uses the standard operations of disjoint sets data structures in the way shown by the following pseudocode.

```
procedure CONNECTED-COMPONENTS(G)
    for each vertex v ∈ G.V do
        MAKE-SET(v)
    for each edge (u, v) ∈ G.E do
        if FIND-SET(u) ≠ FIND-SET(v) then
            UNION(u, v)
```

It iterates on the vertices, creating single-sized sets for each of them, to then join those connected through an edge.

In the end, all edges will have been checked, and the family of sets maintained by the data structure will correspond exactly to the connected components of the graph.

2.2 Linked lists

A **linked list** is a dynamic data structure where its elements, the nodes, are stored in different non-contiguous memory locations. This is rendered possible because of the two attributes maintained by these objects: the actual **data** stored in the node and the pointer to the **next** node in the sequence.

The structure keeps a logical order based on these two pointers: the **head**, representing the first element of the sequence, and the **tail**, the last.

These are the fundamental operations to be done on such a structure:

- **search**: to find a specific element, the starting point must be the head, following each next pointer until it reaches the target.

- **insertion**: to add a node, only its next pointer and that of its new preceding must change.
- **deletion**: to remove a node, the node pointed by its next pointer must become its previous' next.

2.2.1 Implementation of disjoint sets as linked lists

For the purpose of this experiment, we will see how linked lists can be used to implement disjoint sets.

Each set is represented by its own linked list, structured as described above.

The representative of a set is the element of the set stored in the first object of the list: the head.

Here, a brief description of the classic operations of disjoint sets in this particular application:

- **make_set(x)**: a new linked list is created, its only object being x;
- **find_set(x)**: returns the head of the linked list x belongs to;
- **union(x,y)**: y's linked list gets appended onto x's.

Weighted union heuristic Compared to the other two methods, *union* is the most costly, due to the need to change the representative of the appended list.

This cost can be contained by choosing which list to append to which other based on its length.

The **weighted union heuristic** chooses the shortest of the two. This can be achieved by keeping a **size** attribute for every set object.

2.3 Forests of rooted trees

A **tree** is an undirected connected acyclic graph, used as a data structure to store data.

A **rooted tree** is a tree where a specific node, the root, gives a direction for all other edges, establishing parent-child relationships between all the nodes.

These relations are kept by the nodes, by means of the parent pointer set as attribute. The root, which by definition does not descend from any node, keeps itself in this pointer.

A **forest of rooted trees** is a disjoint union of rooted trees such that the trees do not share nodes or edges.

2.3.1 Implementation of disjoint sets as rooted trees

In order to conduct this experiment, we will see an implementation of disjoint sets as a forest of rooted trees.

Each set is represented by a tree whose nodes contain the elements of the set.

The representative of a set is the first node of the tree associated to the set: the root.

Here, a brief description of the classic operations of disjoint sets in this particular application:

- **make_set(x)**: creates a tree with a single node;
- **find_set(x)**: it follows the parent pointers until it reaches the root, tracing the so-called "search path";
- **union(x,y)**: makes it so that the root of y's tree's points to the one of x's tree.

Path compression heuristic In this case, the most costly method is $find_set$, which inevitably impacts on *union*'s execution, seeing as the latter makes use of the roots to join two sets together.

One of the possible ways to improve the efficiency of *union* is to apply the **path compression heuristic** to $find_set$, which consists in making each node of the tree directly point to the root while walking on the search path.

2.4 Assumptions and hypotheses

As we saw before [section 2.1.2], the time complexity of the *connected_components* method depends on all three functions that can be performed on a disjoint set structure.

The complexity of the methods is affected by the data structure chosen to implement the disjoint sets.

Linked lists

- $make_set : O(1)$
- $find_set : O(1)$

Basic $union : O(m \cdot n)$

Weighted union $union : O(m + n \cdot logn)$

Forest of rooted trees

- $make_set : O(1)$
- $find_set : O(logn)$

Path compression $union : O(n + f \cdot (1 + log_{2+f/n} n))$

Where m is the total number of operations, n is the number of $make_set()$ operations and f is the number of $find_set()$ operations. From these theoretical analysis of the methods, one can expect that the structures using a heuristic will perform better, but we have no sufficient a priori knowledge to know which of the two, weighted union on linked lists or path compression on rooted trees, will actually be more efficient.

3 Code documentation

3.1 Content outline and interaction between modules

The code is divided into five modules:

- ***graph.py***: implements the graph data structure, allowing the generation of graphs.
- ***linked_lists.py***: implementation of disjoint sets through linked lists.
- ***forest.py***: implementation of disjoint sets through rooted trees.
- ***test.py***: provides the methods to conduct experiments on the performance of different implementations of disjoint sets, using the different implementations of the *connected_components* algorithm found in the *linked_list.py* and $forset.py$ modules.
- ***main.py***: executes the tests and handles the generation of plots to visualise the results.

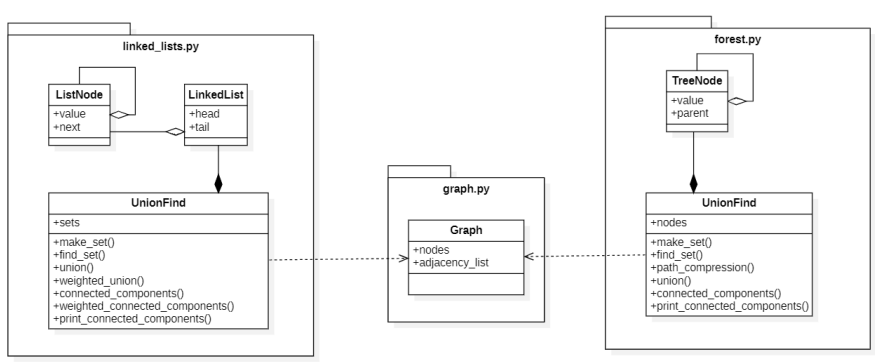

Figure 1: Class diagram

3.2 Analysis of implementation choices

- **Nodes and edges**:
  nodes: unique integers.
  edge: list of two integers.

- **Graph generation**:
  input: number of nodes and edges.
  nodes: sequentially generated to guarantee uniqueness, and kept as a list.
  edges: randomly generated and kept as an adjacency list.

3.3 Description of implemented methods

Functionality of each method from every class.

module *graph.py*

- class *Graph*:
  - ***__init__()*** Constructor. Input: the number of nodes and the number of edges. Generates an undirected randomised graph, using the *random* Python module to initialise the edges.

module *linked_lists.py*

- class *ListNode*
  - ***__init__()*** Constructor. Input: the value to be stored in the attribute *value*. Sets *next* as None.
- class *LinkedList*
  - ***__init__()*** Constructor. Input: a node. Creates a linked list made up of a single node.
- class *UnionFind*
  - ***__init__()*** Constructor. Creates an empty dictionary to keep the linked lists in.
  - ***make_set()*** Input: integer value. Creates a set as a linked list consisting of one node.
  - ***find_set()*** Input: integer value. Returns the head of the linked list to which the node associated with the value belongs.
  - ***union()*** Input: two integer values. Appends the linked list associated with the second value to the one to which the first one belongs, if the nodes with the given value belong to different linked lists.
  - ***weighted_union()*** Input: two integer values. Appends the smaller list of the two associated with the integers, to the bigger one.

- ***connected_components()*** Input: an undirected graph. Finds the connected components of the graph, by building them as disjoint sets. Uses *union*().
- ***weighted_connected_components()*** Input: an undirected graph. Finds the connected components of the graph, by building them as disjoint sets. Uses *weighted_union*().
- ***print_connected_components()*** prints the connected components of the given graph. Can only be used after *connected_components*() or *weighted_connected_components*() have been used on a graph. Used only for debugging purposes.

module *forest.py*

- class *TreeNode*
  - ***__init__()*** Constructor. Input: integer value. Stores the integer in the attribute *value* and sets *parent* as itself.
- class *UnionFind*
  - ***__init__()*** Constructor. Creates an empty dictionary to keep the nodes in.
  - ***make_set()*** Input: integer value. Creates a set, i.e. a tree consisting of one node.
  - ***find_set()*** Input: integer value. Returns the root of the tree to which the node associated with the value belongs, by calling *path_compression*().
  - ***path_compression()*** Input: a node. Recursively changes the parent of each node until every node in the tree has the root as its parent, starting from the given node. Returns the root.
  - ***union()*** Input: two integer values. Links the root of the tree associated with the second value to the root of the other. The two roots are found by calling *find_set*().

module *test.py*

- ***test_lists()*** Input: undirected graph. Times the performance of *connected_components*() from the *linked_lists.py* module, using *perf_counter*() from the Python *time* module.
- ***test_weighted_lists()*** Input: undirected graph. Times the performance of *weighted_connected_components*() from the *linked_lists.py* module, using *perf_counter*() from the Python *time* module.
- ***test_trees()*** Input: undirected graph. Times the performance of *connected_components*() from the *forest.py* module, using *perf_counter*() from the Python *time* module.

- ***create_plot()*** Input: a dictionary such that its keys are strings describing the algorithm used and the corresponding values are lists of the average recorded execution times; a list of values for the x axis of the plot; a string containing the label for the x axis; a string for the title of the plot. Shows a plot containing the pace of all three algorithms on the same graphic.

4 Tests and results

This section is a detailed analysis of the tests conducted and their results, in light of which we will again discuss previous theoretical notions to see whether the outcomes of the experiments are coherent with our expectations or not.

4.1 Data

The experiment analyses the behaviour of the three algorithms in two different ways, for it focuses on two parameters.

- The dimension of the graph, intended as the number of nodes. The studied values of this parameter are saved into a list in the *main.py* module.

```
nodes = [50, 500, 1000, 1500, 2000, 3000, 4000,
    6000, 8000, 10000, 15000]
```

- The density of the graph, intended as the ratio between the number of edges compared to the number of nodes. The studied values of this parameter are saved into a list in the *main.py* module.

```
edges_nodes_ratio = [1, 10, 20]
```

These values were empirically chosen, according to the computational possibilities of the device on which the tests were run.

The density of the graph is fixed as its dimension varies, according to its domain, as defined above.

The variable containing the specific value of the number of nodes can be found in the code as n, while the variable that contains the ratio value of the test is called r.

A hyper-parameter of the test is the number of iterations per scenario to repeat. At each execution a new graph to which the algorithm is applied is created, and since the graph is randomised, this allows for a fairer judgment of the performance overall. The results that are going to be discussed in section 4.3 have been obtained by initialising *main.py*'s variable $ITERATIONS$ with a value of 30.

4.2 Evaluation metrics

The evaluation will be performed using running time as the metric.

The running time of each execution of an algorithm is obtained as the difference between the end time and the start time, saved using the *perf_counter* method of the *time* Python module.
This is, in fact, the main feature of the test functions, each applied to a specific algorithm, in *test.py*.

```
def test_x():
    #[...]
    start = time.perf_counter()
    #function call for the algorithm
    end = time.perf_counter()
    return end - start
```

In *main.py* all these running times are summed into a single variable *x_time*, specific to the algorithm, to compute the statistical average of this measure on the number of executions, dictated by $ITERATIONS$.

```
x_time = 0
for i in range(ITERATIONS):
    #function call to the test method
    x_time += x_time
x_time /= ITERATIONS
```

4.3 Results analysis

The following graphics are the result of the experiments obtained by executing the *main.py* module.

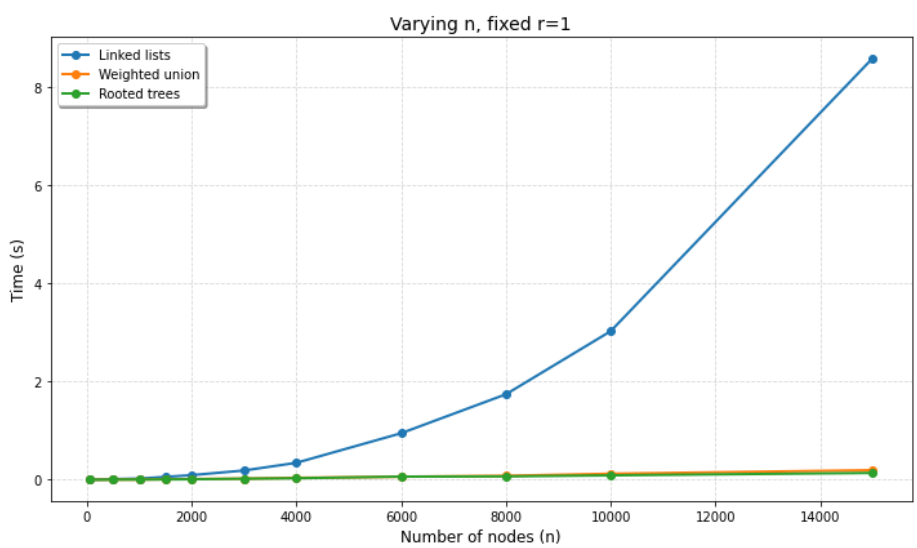

Figure 2: Plot of test case r=1

In this first plot we take into consideration sparse graphs, which are low-density graph.

It is evident how much faster the rooted trees implementation and the weighted union heuristic applied to the linked lists implementation grow, compared to the simple linked lists implementation of the disjoint sets. This shows how the use of heuristics drastically improves the performance of the *connected_components* algorithm. This can be appreciated starting from graphs with 2000 nodes.

In this first test case, we can appreciate near to no difference between the two algorithm with an applied heuristic.

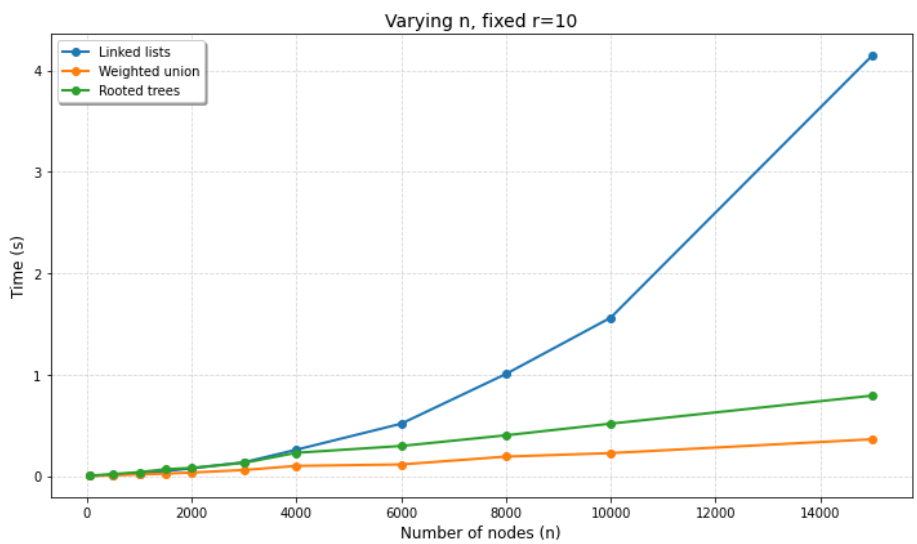

Figure 3: Plot of test case r=10

By augmenting the density, we start to notice a greater robustness on the weighted union heuristic's part.

The curves do still show the distinction between the logarithmic growth of the two heuristics compared to the quadratic pace of the simple linked lists implementation.

The difference in performance, however, starts being appreciated on bigger values of n: 2000 nodes to distinguish the behaviour of the weighted union algorithm from the other two, and 4000 for the simple linked lists implementation's curve to diverge from the rooted trees.

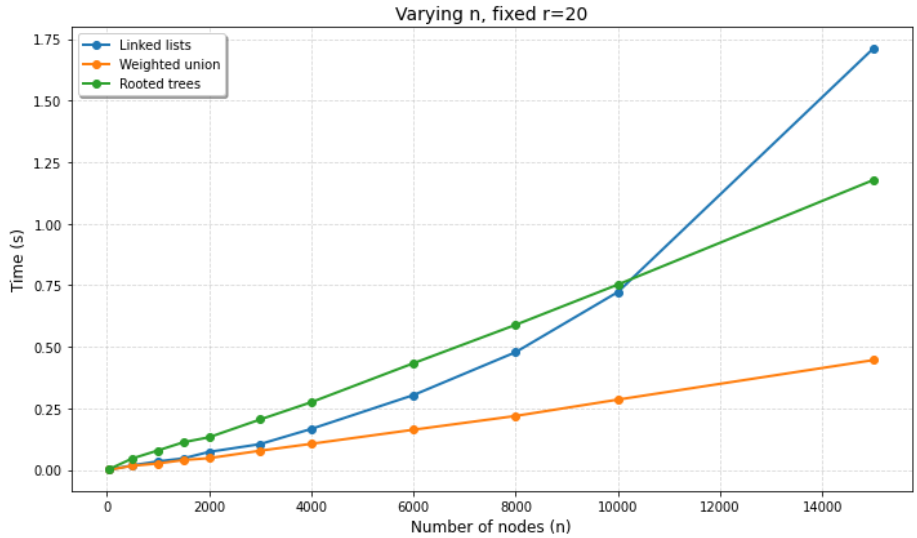

Figure 4: Plot of test case r=20

The situation shown in this case shows more drastic changes.

While the weighted union heuristic keeps its place as the best performative of the three, as far as the other two are concerned, their behaviour has changed.

It is obvious how the grown density of the graph has affected the path compression heuristic more so than it did the others, making it the least efficient out of the three for smaller graphs. Its asymptotic logarithmic growth reveals itself in larger numbers: after 10000 nodes it keeps its steady pace, while the linked list algorithm's curve diverges from the other.

On the other hand, the linked lists implementation's quadratic growth appears to be slowed, as shown by the less steep curve in the plot, making its curve follow that of the weighted union heuristic until the graphs' dimension reaches 2000 nodes, where the two start having two distinctively different paces.

5 Conclusion

The data gathered from the tests described and analysed in the previous sections provide enough information to draw a conclusion regarding the efficiency of the disjoint data structures evaluated in this experiment.

The fundamental insight is that there is not an absolute best algorithm in terms of performance; rather the empirical behaviour of the algorithms is strongly dependent on the density of the graph (r).

This dependency becomes clear when examining the distinction between the three test scenarios. Regarding sparse graphs (r=1), the application of heuristics has a quite evident positive effect, confirming the asymptotic expected evolution.

However, for smaller values of n (up to 2000 nodes), the behaviour of all three algorithms is nearly identical, with the curves almost perfectly overlapping.

As density increases, the positive impact of heuristics is not always guaranteed. While the theoretical analysis suggests heuristics should strictly improve efficiency, the experimental results for dense graphs (r=20) show that the rooted trees implementation using path compression worsens significantly, becoming the least efficient method. On the other hand, the simple linked list implementation handles increased density surprisingly well.

Ultimately, while the connected components algorithm is affected by increasing density across all implementations, the weighted union heuristic proves to be the most robust overall. Even though it is slowed by increasing density, it maintains the most consistent performance advantage across all tested scenarios compared to the variability of path compression and the inefficiency of simple linked lists in large, sparse graphs.

6 References

[1] *Thomas H.Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein (2009) Introduzione agli algoritmi e strutture dati Terza Edizione, McGraw Hill.*